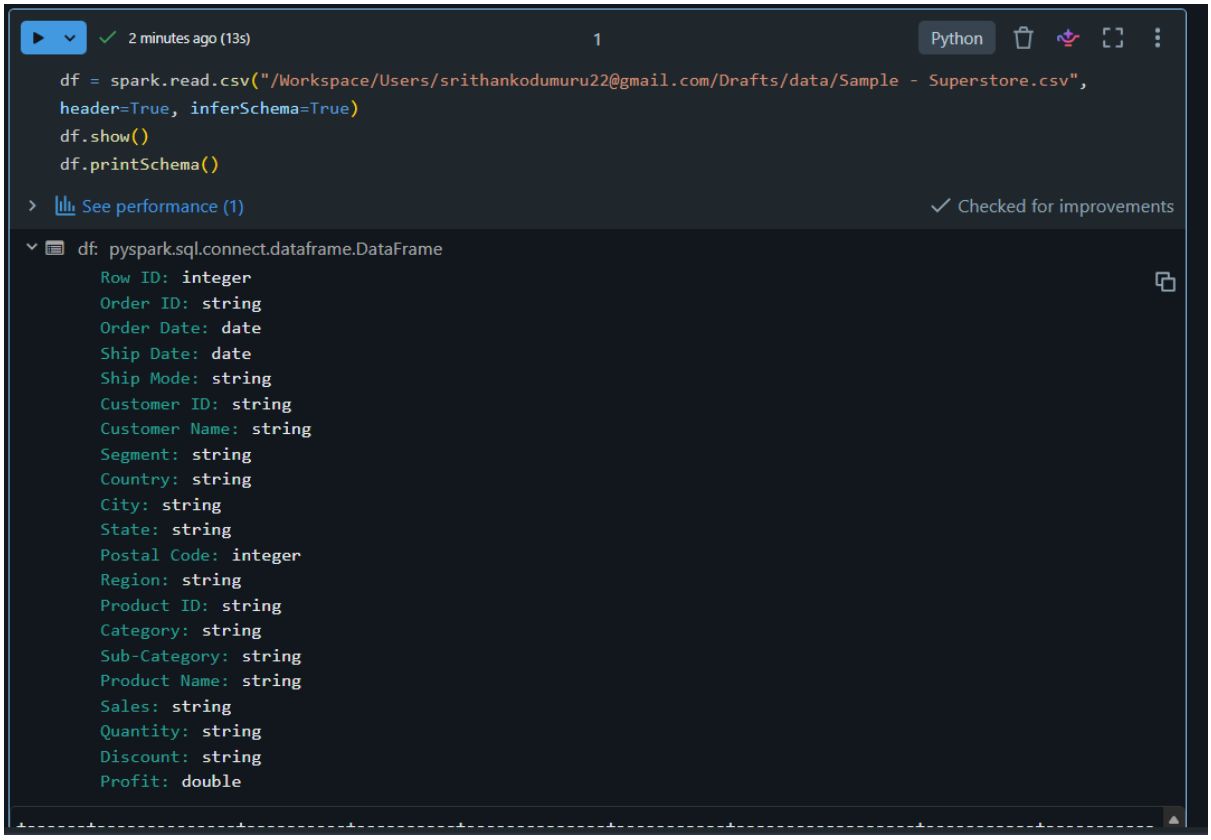
Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer: The Driver is the main part of a Spark application. It creates the Spark session, manages the program, and sends tasks to the executors. The Cluster Manager manages the cluster resources and assigns them to the Spark application. The Executors are the worker processes that run the tasks given by the Driver and process the data. They also store intermediate data during execution.

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer: Spark does not execute the code immediately. It first stores all the transformations and waits until an action like show() or collect() is called. This helps Spark optimize the execution and avoid unnecessary work, making data processing faster and more efficient.

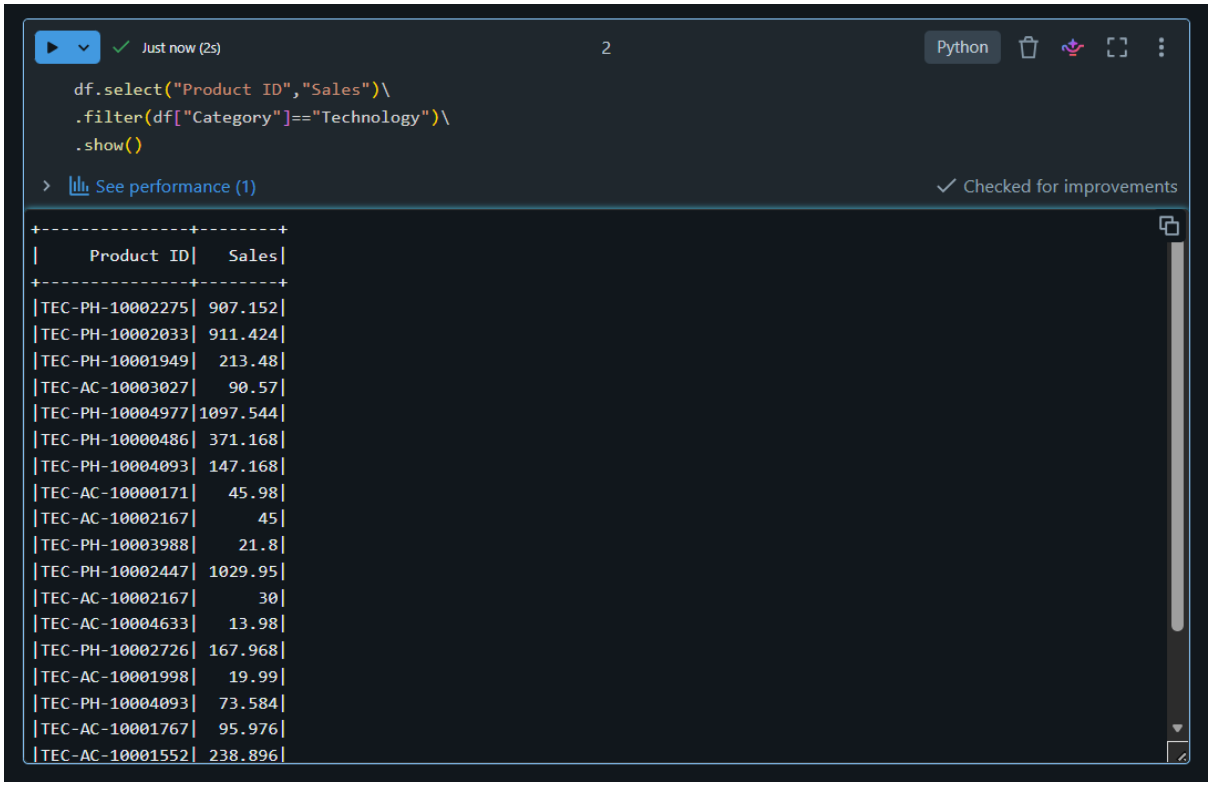
Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.



Q4. What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

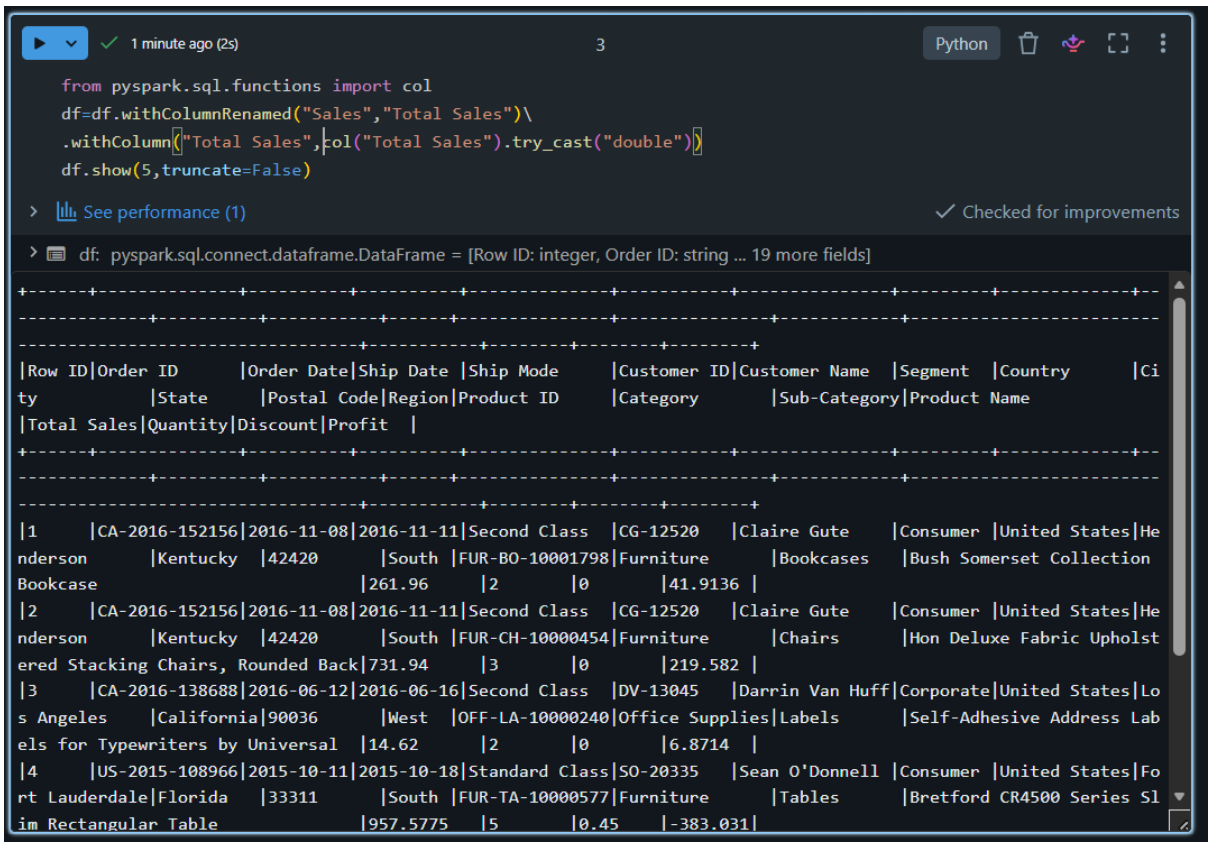
Answer : CSV stores data row by row, while Parquet stores data column by column. Parquet is faster because Spark reads only the required columns instead of the whole file. It also uses compression, which reduces storage space and improves performance.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.



Note: Superstore dataset does not have an Electronics category. It uses categories like Technology, Furniture, and Office Supplies, so Technology is the correct value to filter.

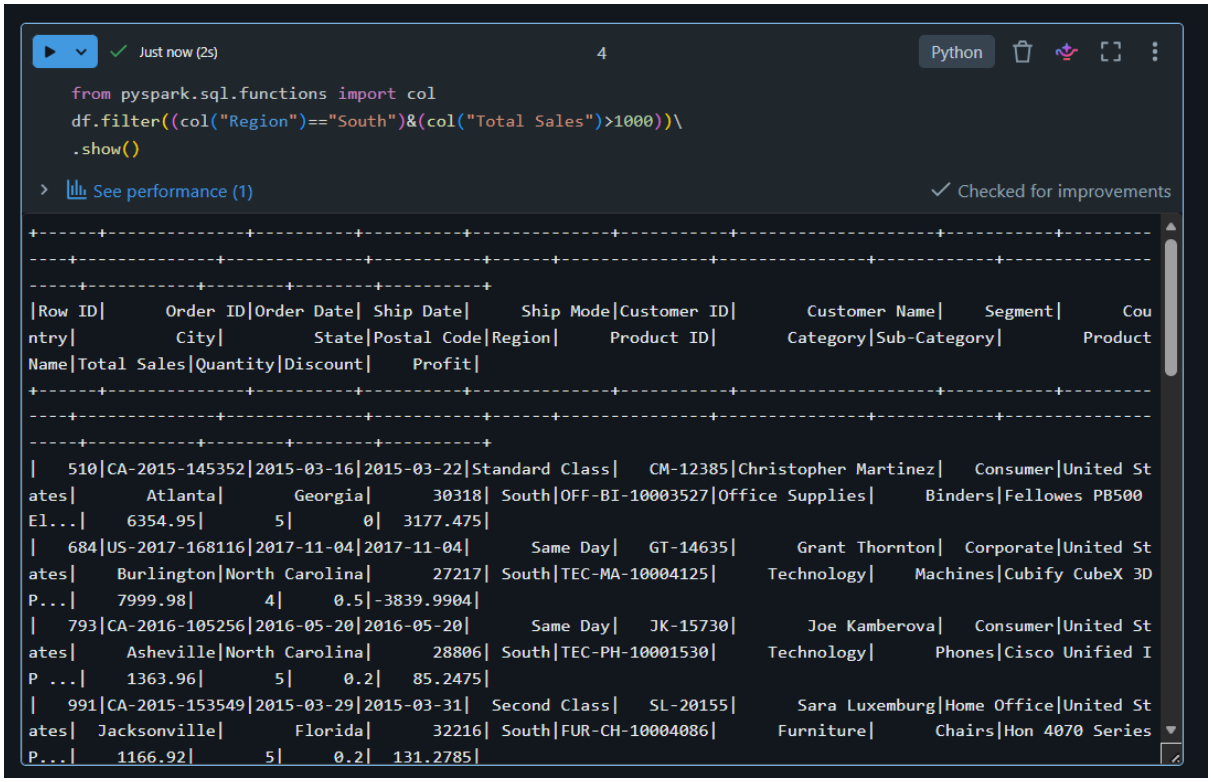
Q6. Write the code to revise a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.



Q7. How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Answer : Spark keeps a record of all the transformations in a Lineage Graph (DAG). If a worker node fails and some data is lost, Spark uses this graph to recompute only the missing data instead of processing the entire dataset again. This provides fault tolerance and saves time.

Q8. Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.



Q9. Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Answer: Predicate Pushdown is a feature in Parquet where Spark applies filter conditions while reading the file. It reads only the required data instead of the entire dataset. This reduces memory usage, decreases disk I/O, and improves performance.

Q10. Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).

Just now (2s) 5 Python

```python
from pyspark.sql.functions import col

df=df.withColumn("Final Price",col("Total Sales")*1.18)
df.show(5)
```

See performance (1) ✓ Checked for improvements

df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: integer, Order ID: string ... 20 more fields]

```
+------+--------------+----------+----------+--------------+-----------+----------------+---------+-------------+--
------------+----------+-----------+------+---------------+---------------+------------+--------------------+----
-------+--------+--------+--------+-----------------+
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|   Customer Name|  Segment|      Country|
City|     State|Postal Code|Region|     Product ID|       Category|Sub-Category|        Product Name|Total Sales|Q
uantity|Discount|  Profit|       Final Price|
+------+--------------+----------+----------+--------------+-----------+----------------+---------+-------------+--
------------+----------+-----------+------+---------------+---------------+------------+--------------------+----
-------+--------+--------+--------+-----------------+
|     1|CA-2016-152156|2016-11-08|2016-11-11|  Second Class|   CG-12520|     Claire Gute| Consumer|United States|
Henderson|  Kentucky|      42420| South|FUR-BO-10001798|      Furniture|   Bookcases|Bush Somerset Col...|      26
1.96|       2|       0| 41.9136|309.11279999999994|
|     2|CA-2016-152156|2016-11-08|2016-11-11|  Second Class|   CG-12520|     Claire Gute| Consumer|United States|
Henderson|  Kentucky|      42420| South|FUR-CH-10000454|      Furniture|      Chairs|Hon Deluxe Fabric...|      73
1.94|       3|       0| 219.582|          863.6892|
|     3|CA-2016-138688|2016-06-12|2016-06-16|  Second Class|   DV-13045|Darrin Van Huff|Corporate|United States|
Los Angeles|California|      90036|  West|OFF-LA-10000240|Office Supplies|      Labels|Self-Adhesive Add...|
14.62|       2|       0|  6.8714|           17.2516|
|     4|US-2015-108966|2015-10-11|2015-10-18|Standard Class|   SO-20335| Sean O'Donnell| Consumer|United States|Fo
rt Lauderdale|   Florida|      33311| South|FUR-TA-10000577|      Furniture|      Tables|Bretford CR4500 S...|     9
57.5775|       5|    0.45|-383.031|        1129.94145|
```

Q11. What is the difference between Transformations and Actions? Provide two examples of each.

Answer: Transformations are operations that change the data or create a new dataframe , but Spark does not execute them immediately. It waits until an action is called. Actions are operations that execute all the transformations and return the result.

Examples of Transformations: filter(), select()

Examples of Actions: show(), collect()

Q12. Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

09:17 AM (9s) 24

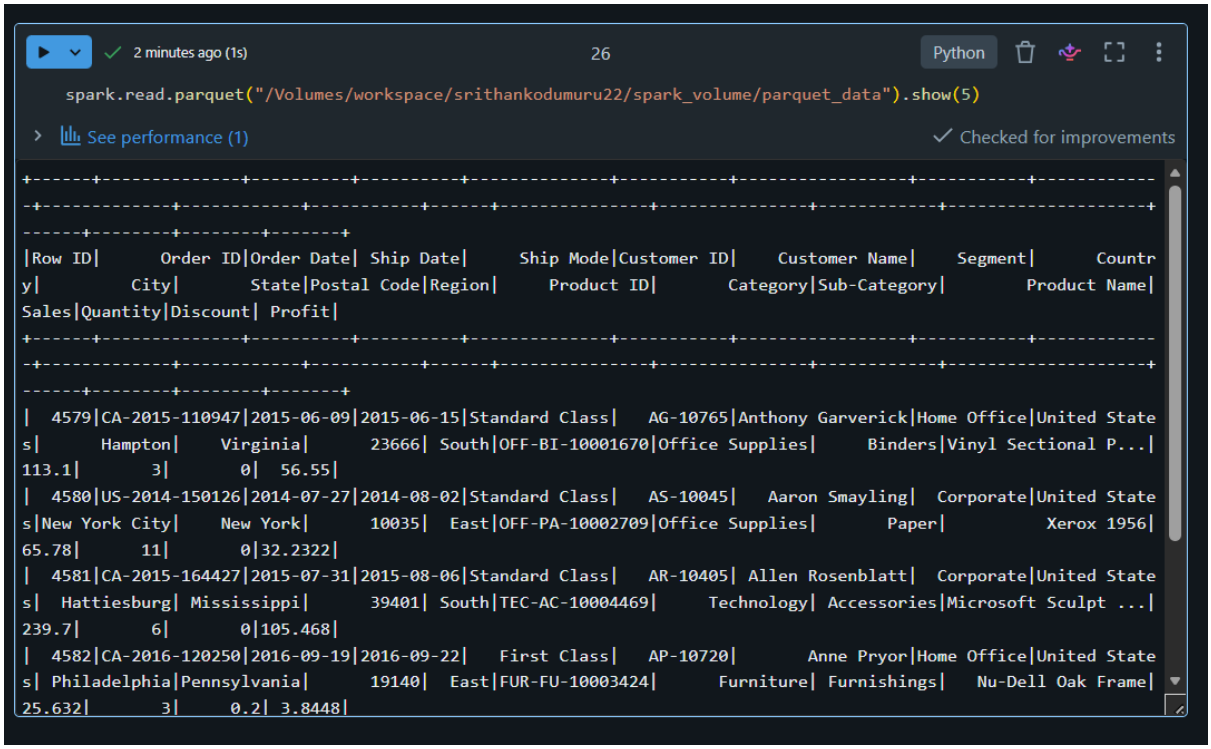
```python
df.write.mode("overwrite").parquet("/Volumes/workspace/srithankodumuru22/spark_volume/parquet_data")
```

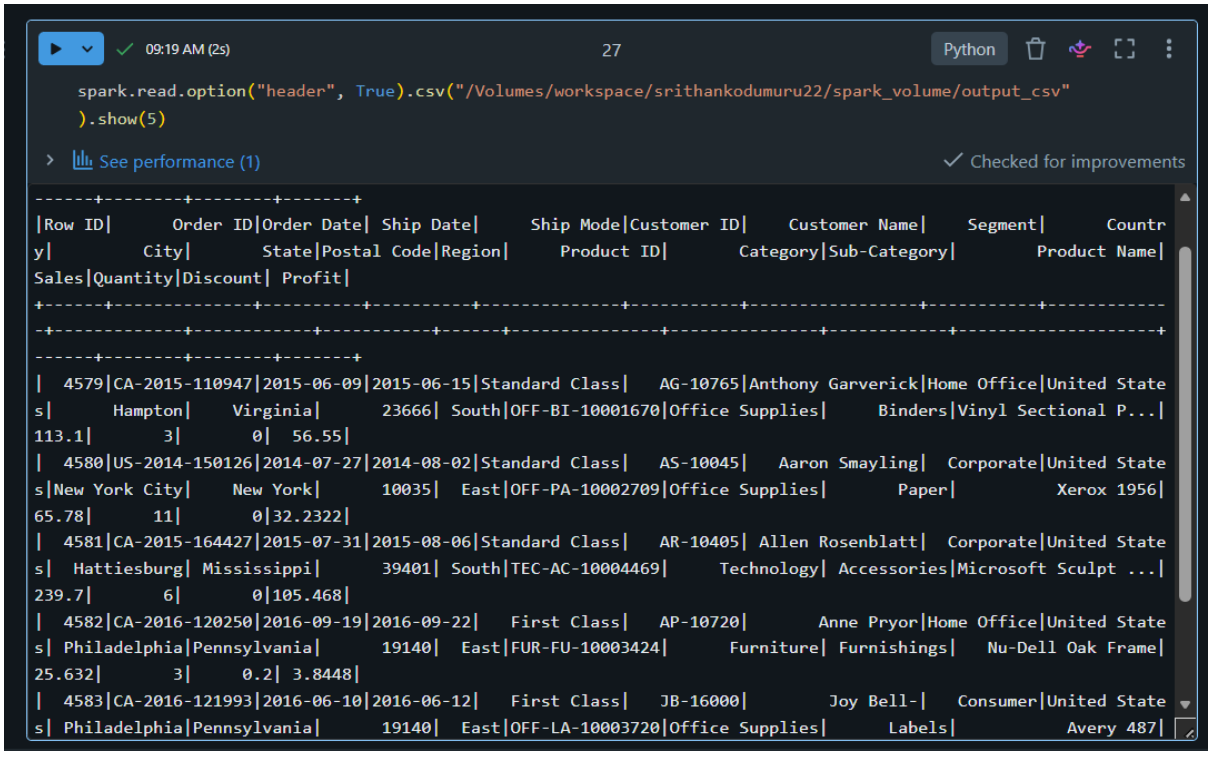
See performance (1) ✓ Checked for improvements

09:18 AM (3s) 25

```python
spark.read.parquet("/Volumes/workspace/srithankodumuru22/spark_volume/parquet_data").\
filter("`Customer ID` IS NOT NULL").write.mode("overwrite")\
.option("header", True) \
 .csv("/Volumes/workspace/srithankodumuru22/spark_volume/output_csv")
```

See performance (1) ✓ Checked for improvements



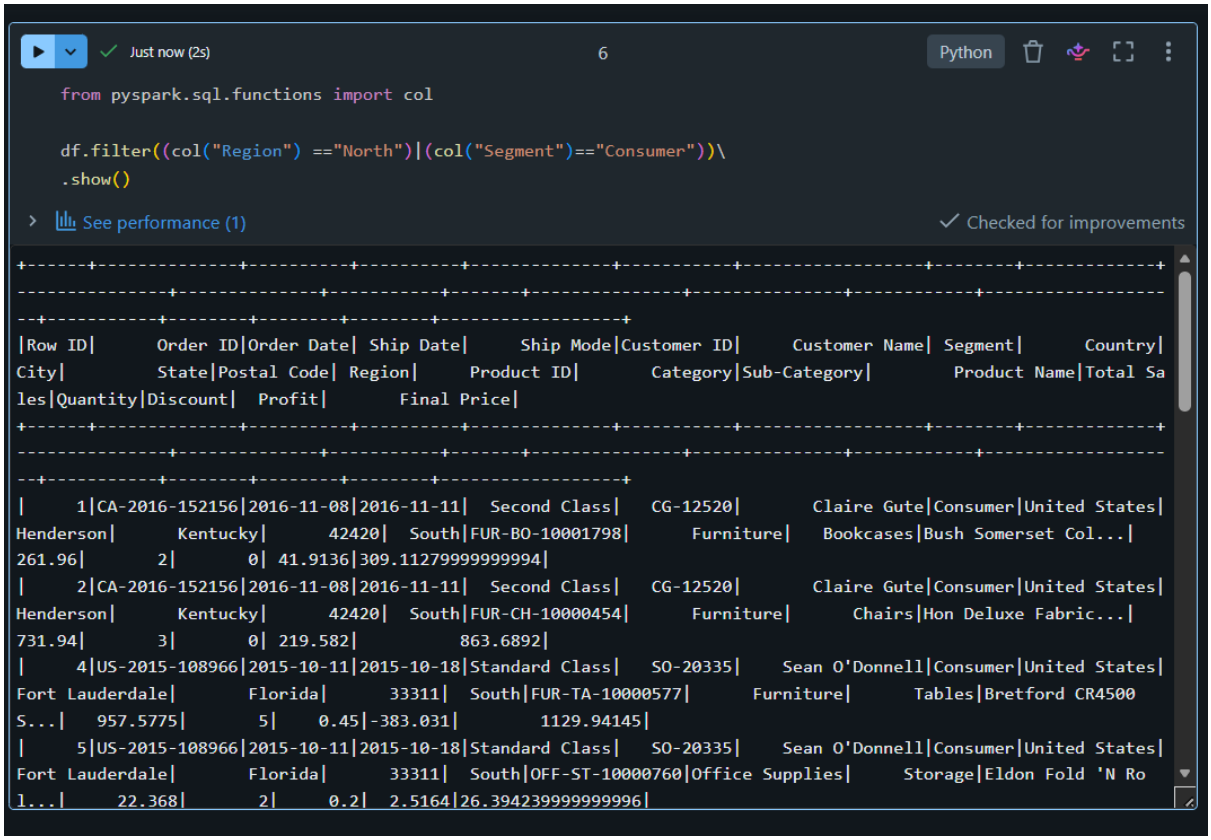


Implementation: Initially, I encountered a storage permission issue while saving the output files. After identifying the correct writable location, I created and used a Databricks Volume (spark_volume). I successfully saved the dataset as a Parquet file, read it back, filtered the required records, and saved the final output as a CSV file.

Q13. In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Answer : In Client Mode, the Driver program runs on the local machine where the Spark application is submitted. In Cluster Mode, the Driver runs inside the cluster along with the executors. Client Mode is mainly used for development and testing, while Cluster Mode is preferred for running production applications.

Q14. Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.



Q15. When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

Answer :.show(5) displays only the first 5 rows of the dataset, so it uses very little memory. On the other hand, .collect() loads the entire dataset into the Driver's memory. If the dataset is very large, it can cause memory issues or even crash the application. That's why .show(5) is safer for exploring large datasets.

Workflow

Read CSV Dataset

↓

Check Schema

↓

Select Required Columns

↓

Filter Data

↓

Modify DataFrame

↓

Handle Null Values

↓

Save Output